TRIBHUVAN UNIVERSITY
Institute of Engineering | BCT Program

OPERATING SYSTEM (CT 656)

CHAPTER 4: MEMORY MANAGEMENT

Complete Exam Answer Guide | 6 Hours | 10 Marks

Topic No.	Syllabus Content
4.1	Memory address, Swapping and Managing Free Memory Space
4.2	Resident Monitor
4.3	Multiprogramming with Fixed Partition
4.4	Multiprogramming With Variable Partition
4.5	Multiple Base Register
4.6	Virtual Memory Management
4.6.1	Paging
4.6.2	Segmentation
4.6.3	Paged Segmentation
4.7	Demand Paging
4.8	Performance
4.9	Page Replacement Algorithms
4.10	Allocation of Frames
4.11	Thrashing

Question 1: Demand Paging & Page Replacement Algorithms

Q: What is Demand Paging? Consider the following page reference string: 2,3,4,2,1,3,7,5,4,3,1,5. Find how many page faults occur according to OPTIMAL, LRU (least recently used) and LFU (least frequently used) page replacement algorithm assuming 3-page frames.

Asked in: 2080 Chaitra [1+6], 2079 Jestha [1+5], 2081 Chaitra [1+6]

1.1 Demand Paging

Demand paging is a virtual memory management technique where pages are loaded into memory only when they are needed (demanded), not in advance. It is a form of lazy loading.

Key Concepts:

- Page Fault: Occurs when a requested page is not in memory — OS must load it from disk.
- Valid-Invalid Bit: Each page table entry has a bit indicating if page is in memory (1) or not (0).

- Advantages: Less memory needed, faster startup, supports large address spaces.
- Disadvantages: Page fault overhead, thrashing risk.

1.2 FIFO Page Replacement (Reference: 2,3,4,2,1,3,7,5,4,3,1,5 | 3 Frames)

In FIFO, the oldest page in memory is replaced first.

Reference String: 2, 3, 4, 2, 1, 3, 7, 5, 4, 3, 1, 5

Page Ref	2	3	4	2	1	3	7	5	4	3	1	5
Frame 1	2	2	2	2	1	1	1	5	5	5	5	5
Frame 2	-	3	3	3	3	3	7	7	4	4	4	4
Frame 3	-	-	4	4	4	4	4	4	4	3	1	1
Fault?	F	F	F	-	F	-	F	F	F	F	F	-

Total FIFO Page Faults = 9 (marked F above)

1.3 LRU Page Replacement (3 Frames)

In LRU, the page that has not been used for the longest time is replaced.

Page Ref	2	3	4	2	1	3	7	5	4	3	1	5
Frame 1	2	2	2	2	2	2	7	7	4	4	1	1
Frame 2	-	3	3	3	1	1	1	5	5	3	3	3
Frame 3	-	-	4	4	4	3	3	3	3	3	3	5
Fault?	F	F	F	-	F	F	F	F	F	F	F	F

Total LRU Page Faults = 10 (Note: LRU replaces 2 at step 5 since 2 was used at step 1, oldest)

1.4 Optimal Page Replacement (3 Frames)

In Optimal, the page that will not be used for the longest time in the future is replaced.

Page Ref	2	3	4	2	1	3	7	5	4	3	1	5
Frame 1	2	2	2	2	2	2	2	2	4	4	1	1
Frame 2	-	3	3	3	3	3	3	5	5	5	5	5
Frame 3	-	-	4	4	1	1	7	7	7	3	3	3
Fault?	F	F	F	-	F	-	F	F	F	F	F	-

Total Optimal Page Faults = 9

Note: Optimal is theoretical (requires future knowledge) — used as benchmark.

Question 2: Internal vs External Fragmentation & Partitioning

Q: Differentiate between internal and external fragmentation. Suppose that we have memory of 1000 KB with 5 partitions of size 150 KB, 200 KB, 250 KB, 100 KB, and 300 KB. Where will processes A (175 KB) and B (125 KB) be loaded, if we used Best-Fit and Worst-Fit strategy?

- **Asked in: 2079 Chaitra [5], 2081 Ashwin [4+6]**
- **Marks: [5] marks**

2.1 Internal vs External Fragmentation

Feature	Internal Fragmentation	External Fragmentation
Definition	Wasted space inside allocated partition	Free memory scattered in small unusable pieces
Cause	Fixed partition larger than process size	Variable partition allocation/deallocation
Location	Inside allocated memory block	Outside allocated blocks (between them)
Solution	Smaller partition sizes	Compaction, Paging, Segmentation
Example	32KB process in 64KB block wastes 32KB	10KB+5KB+8KB free but 20KB cannot be allocated

2.2 Solved: Memory Allocation Strategies

Memory partitions: 150 KB, 200 KB, 250 KB, 100 KB, 300 KB

Process A = 175 KB, Process B = 125 KB

Strategy	Process A (175KB)	Process B (125KB)	Reason
Best-Fit	200 KB (25 KB waste)	150 KB (25 KB waste)	Smallest adequate partition
Worst-Fit	300 KB (125 KB waste)	250 KB (125 KB waste)	Largest partition first

Best-Fit minimizes waste per allocation; Worst-Fit leaves large enough holes for future processes.

Question 3: Paging — Logical/Physical Address Calculation

Q: Define page fault and demand paging. Consider logical address space of 8 pages of 1024 words, each mapped onto a physical memory of 32 frames. (a) How many bits are in the logical address and physical address? (b) How will paging be done?

- **Asked in: 2078 Chaitra [4+6]**
- **Marks: [4+6] = 10 marks**

3.1 Definitions

Page Fault: An interrupt that occurs when a process tries to access a page not currently in memory. The OS then loads the required page from secondary storage (disk) into a free frame.

Demand Paging: A virtual memory technique where pages are loaded only when needed. The OS uses a 'valid-invalid' bit in the page table to track which pages are in memory.

3.2 Calculating Address Bits

Given:

- Logical address space: 8 pages of 1024 words each
- Physical memory: 32 frames

Logical Address Calculation:

- Number of pages = 8 → Page number bits = $\log_2$ (8) = 3 bits
- Page size = 1024 words → Offset bits = $\log_2$ (1024) = 10 bits
- Total logical address = 3 + 10 = 13 bits

Physical Address Calculation:

- Number of frames = 32 → Frame number bits = $\log_2$ (32) = 5 bits
- Frame size = Page size = 1024 words → Offset bits = 10 bits
- Total physical address = 5 + 10 = 15 bits

Address Type	Components	Total Bits
Logical Address	Page No. (3) + Offset (10)	**13 bits**
Physical Address	Frame No. (5) + Offset (10)	**15 bits**

3.3 How Paging Works

1. CPU generates logical address → split into Page Number (p) and Offset (d).
2. Page table indexed by p gives Frame Number (f).
3. Physical address = f concatenated with d.
4. TLB (Translation Lookaside Buffer) caches recent page table entries for fast translation.

Question 4: TLB and Page Fault Calculation

Q: What do you mean by TLB? Consider the following page references: 0 1 8 7 0 3 1 0 2 8 1 7 3 0 1 8 7 0 3. Calculate the number of page faults according to Optimal, FIFO and LRU algorithms. Assume available frames = 3.

- **Asked in: 2081 Chaitra [2+6]**
- **Marks: [2+6] = 8 marks**

4.1 TLB — Translation Lookaside Buffer

TLB is a small, fast cache (associative memory) that stores recently used page-table entries to speed up virtual-to-physical address translation.

How TLB works:

- CPU generates logical address with page number p
- TLB is searched simultaneously for p
- TLB Hit: Frame number found directly → fast access
- TLB Miss: Page table in memory consulted → TLB updated

Effective Access Time (EAT) = α × (cache + memory) + (1-α) × (cache + 2×memory) where α = TLB hit ratio

4.2 FIFO Page Replacement (String: 0,1,8,7,0,3,1,0,2,8,1,7 | 3 Frames)

Using first 12 references for demonstration. FIFO replaces the oldest page.

Ref	0	1	8	7	0	3	1	0	2	8	1	7
F1	0	0	0	7	7	7	7	7	2	2	2	2
F2	-	1	1	1	1	3	3	3	3	8	8	8
F3	-	-	8	8	8	8	1	1	1	1	1	7
Fault?	**F**	**F**	**F**	**F**	**-**	**F**	**F**	**-**	**F**	**F**	**-**	**F**

Total FIFO Faults = 9

4.3 Comparison Summary

Algorithm	Page Faults	Characteristic
OPTIMAL	Minimum possible	Theoretical benchmark
LRU	Usually close to optimal	Replaces least recently used
FIFO	~9 (can suffer Belady's)	Replaces oldest loaded

Question 5: Virtual Memory — Segmentation & Paged Segmentation

Q: What is Associative memory? Explain Virtual Memory and Thrashing. Write the difference between Coalescing and Compaction.

- **Asked in: 2079 Jestha [1+1+2], 2081 Ashwin [1+1+2]**
- **Marks: [1+1+2] = 4 marks**

5.1 Associative Memory (Content-Addressable Memory)

Associative memory (also called Content-Addressable Memory or CAM) is a type of memory that can be accessed by its content rather than by its address. In TLB, all entries are searched simultaneously for a matching page number — this parallel search is what makes TLB fast.

5.2 Virtual Memory

Virtual memory is a technique that allows processes to use more memory than physically available by using disk storage as an extension of RAM. Key mechanisms:

- Demand Paging: Pages loaded only when needed
- Page Tables: Map virtual addresses to physical frames
- Secondary Storage: Backing store for pages not in RAM

5.3 Thrashing

Thrashing occurs when a process spends more time paging (swapping pages in/out) than executing. The CPU utilization drops dramatically.

Cause: Too many processes in memory, insufficient frames allocated per process.

Solution: Working Set Model — allocate frames based on a process's working set (pages actively used in a time window).

5.4 Compaction vs Coalescing

Feature	Compaction	Coalescing
Definition	Moves all occupied memory to one end to create one large free block	Merges two adjacent free blocks into one larger free block
When done	Periodically or when large allocation needed	Immediately when a block is freed
Overhead	Very high (copies entire processes)	Low (just adjusts free list pointers)
Result	Single large contiguous free space	Larger but possibly fragmented free space

Question 6: Page Replacement — Second Chance (Clock) Algorithm

Q: How does the second chance page replacement algorithm improve FIFO? Consider the following page reference string: 2, 3, 4, 2, 1, 3, 7, 5, 4, 3, 1, 5. Find how many page faults occur according to OPTIMAL, LRU and LFU page replacement algorithm assuming 3 page frames.

- **Asked in: 2080 Ashwin [2+5], 2079 Chaitra [2+5]**
- **Marks: [2+5] = 7 marks**

6.1 Second Chance Algorithm vs FIFO

The Second Chance (Clock) algorithm improves FIFO by giving pages a second chance before replacing them:

- Each page has a reference bit (R) set to 1 when accessed.
- When a page must be replaced, FIFO's oldest page is checked.
- If R=1: Clear R to 0, advance pointer (give second chance), skip this page.
- If R=0: Replace this page — it had its second chance and was not used.

Advantage: Pages that are frequently used get a 'second chance' and are unlikely to be evicted even if they're old.

6.2 LFU Page Replacement (String: 2,3,4,2,1,3,7,5,4,3,1,5 | 3 Frames)

LFU replaces the page with the lowest use frequency. Ties broken by oldest page.

Page Ref	2	3	4	2	1	3	7	5	4	3	1	5
Frame 1	2	2	2	2	2	2	2	5	4	3	1	5
Frame 2	-	3	3	3	3	3	3	3	3	3	3	3
Frame 3	-	-	4	4	1	1	7	7	7	7	7	7
Fault?	**F**	**F**	**F**	**-**	**F**	**-**	**F**	**F**	**F**	**F**	**F**	**F**

Total LFU Page Faults = 10 (LFU replaces page with lowest frequency on each fault)

Question 7: Frame Allocation & Thrashing (Short Notes)

Q: Write short notes on: (a) Belady's Anomaly (b) Thrashing (c) Compaction and Coalescing

Asked in: 2082 Kartik [3×3], 2079 Jestha [2×4], 2079 Ashwin [3×4]

Marks: [3×3] = 9 marks

7.1 Belady's Anomaly

Belady's Anomaly is the counterintuitive phenomenon where increasing the number of page frames can actually result in MORE page faults for certain page replacement algorithms (specifically FIFO).

Example: Reference string 1,2,3,4,1,2,5,1,2,3,4,5

- With 3 frames (FIFO): 9 page faults
- With 4 frames (FIFO): 10 page faults — MORE faults with more frames!

LRU and Optimal algorithms do NOT suffer from Belady's Anomaly — they are called 'stack algorithms'.

7.2 Thrashing

Thrashing is a condition where a computer spends more time paging (swapping) than executing useful work. CPU utilization falls drastically.

Causes:

- Process has too few frames — every instruction causes a page fault
- OS adds more processes to improve CPU utilization — worsens situation

Detection: Monitor page-fault rate. High rate = thrashing.

Solution:

- Working Set Model: Track pages used in last Δ time; allocate exactly those frames
- Page Fault Frequency (PFF) Control: Set upper/lower bound on fault rate

7.3 Allocation of Frames Strategies

Strategy	Description
Equal Allocation	Each process gets equal share: total_frames / num_processes
Proportional Allocation	Frames allocated proportional to process size
Priority Allocation	Higher priority processes get more frames
Global Replacement	Process can steal frames from any other process
Local Replacement	Process can only replace its own frames

Question 8: Memory Management Summary — Key Formulas & Concepts

Q: What are the strategies for memory management? Explain paging with logical and physical address structures. [From various years]

- **Asked in: 2079 Chaitra [5], 2079 Ashwin [3], 2080 Ashwin [2]**
- **Marks: [5] marks**

8.1 Fixed vs Variable Partitioning

Feature	Fixed Partitioning	Variable Partitioning
Partition size	Fixed at system start	Varies based on process size
Fragmentation	Internal fragmentation	External fragmentation
Number of partitions	Fixed	Dynamic
Flexibility	Low	High
Placement	Equal or unequal fixed sizes	Best-fit, First-fit, Worst-fit

8.2 Key Memory Management Formulas

```
Logical Address  = Page Number (p)  + Offset (d)
Physical Address = Frame Number (f) + Offset (d)
Physical Address = Frame_Table[p] × Page_Size + d
Number of Page No. bits = log□(num_pages)
Number of Offset bits   = log□(page_size_in_words)
Page size = Frame size (always equal)
```

8.3 Segmentation

Segmentation divides a program into logical segments (code, stack, data, heap). Each segment has a name/number and length.

- Segment Table: Contains base address and limit (length) for each segment
- Logical Address: <Segment number, offset>
- Physical Address: Base[s] + offset (if offset < limit[s])
- Advantage: Supports protection and sharing at logical level
- Disadvantage: External fragmentation

8.4 Swapping

Swapping is a process where a process in memory is swapped to a backing store (disk) to free up memory for another process, then swapped back in when needed.

- Roll Out/Roll In: Swapping based on priority
- Context Switch time increases if swapping needed
- Major portion of swap time = Transfer time (proportional to memory size)